

시설팀 민원 앱 API 문서

백엔드 API 연동 가이드입니다. 순서대로 연동해주세요!

📌 기본 설정

Base URL

개발: `http://localhost`
운영: `https://[운영서버 도메인]`

인증 방법

로그인 후 발급받은 JWT 토큰을 모든 요청 헤더에 포함해주세요.

`Authorization: Bearer {token}`

토큰 저장

```
// 로그인 성공 시 localStorage에 저장
localStorage.setItem('token', response.token);
localStorage.setItem('user', JSON.stringify(response.member));

// 요청 시 토큰 가져오기
const token = localStorage.getItem('token');
```

📌 공통 응답 형식

성공

```
{
  "message": "성공 메시지",
  "data": { ... }
}
```

실패

```
{
  "message": "오류 메시지"
}
```

HTTP 상태 코드

코드	설명
200	성공
201	생성 성공
400	잘못된 요청
401	인증 실패

코드	설명
403	권한 없음
404	찾을 수 없음
409	중복 (이미 존재)
500	서버 오류

연동 순서 가이드

1단계: 로그인 / 회원가입
2단계: 카테고리 목록 조회
3단계: 민원 목록 / 등록 / 상세
4단계: 공지사항
5단계: 알림
6단계: 내 정보 수정

1 인증 API

아이디 중복 확인

GET /api/auth/check-id/:login_id
인증: 불필요

응답

```
{
  "available": true,
  "message": "사용 가능한 아이디입니다."
}
```

```
{
  "available": false,
  "message": "이미 사용 중인 아이디입니다."
}
```

회원가입

POST /api/auth/register
인증: 불필요

요청 바디

```
{
  "login_id": "hong001",
  "password": "test1234!",
  "name": "홍길동",
  "role": "C",
  "dept": "총무팀",
  "phone": "010-1234-5678"
}
```

role: c (사용자), E (처리자) 처리자(E) 가입 시 dept는 카테고리 목록 API(GET /api/categories/with-total)에서 가져온 값 사용

응답

```
{
  "message": "회원가입이 완료되었습니다. 관리자 승인 후 로그인 가능합니다.",
  "member": {
    "member_id": 1,
    "login_id": "hong001",
    "name": "홍길동",
    "role": "C",
    "dept": "총무팀",
    "phone": "010-1234-5678",
    "is_approved": false
  }
}
```

로그인

POST /api/auth/login
인증: 불필요

요청 바디

```
{
  "login_id": "hong001",
  "password": "test1234!"
}
```

응답

```
{
  "message": "로그인 성공",
  "token": "eyJhbGciOi...",
  "member": {
    "member_id": 1,
    "login_id": "hong001",
    "name": "홍길동",
    "role": "C",
    "dept": "총무팀",
    "phone": "010-1234-5678"
  }
}
```

로그인 성공 시 token 과 member 를 localStorage에 저장해주세요.

오류 케이스

```
{ "message": "관리자 승인 대기 중입니다." } // 미승인 계정
{ "message": "아이디 또는 비밀번호가 올바르지 않습니다." }
```

로그아웃

POST /api/auth/logout
인증: 필요

서버에 요청 후 localStorage에서 token , user 삭제해주세요.

```
localStorage.removeItem('token');  
localStorage.removeItem('user');
```

내 정보 조회

GET /api/auth/me
인증: 필요

응답

```
{  
  "member": {  
    "member_id": 1,  
    "login_id": "hong001",  
    "name": "홍길동",  
    "role": "C",  
    "dept": "총무팀",  
    "phone": "010-1234-5678",  
    "is_approved": true  
  }  
}
```

2 카테고리 API

카테고리 목록 조회 (민원 등록용 - 전체 제외)

GET /api/categories
인증: 불필요

응답

```
{  
  "categories": [  
    { "category_id": 1, "category_name": "건축/영선", "dept": "시설팀" },  
    { "category_id": 2, "category_name": "의료장비", "dept": "물류관리팀" },  
    { "category_id": 3, "category_name": "기계/소방", "dept": "시설팀" },  
    { "category_id": 4, "category_name": "전기/통신", "dept": "시설팀" },  
    { "category_id": 5, "category_name": "보안", "dept": "총무팀" },  
    { "category_id": 6, "category_name": "미화", "dept": "총무팀" }  
  ]  
}
```

카테고리 목록 조회 (처리자 가입용 - 전체 포함)

GET /api/categories/with-total
인증: 불필요

응답

```
{  
  "categories": [  
    { "category_id": 0, "category_name": "전체", "dept": null },  
    { "category_id": 1, "category_name": "건축/영선", "dept": "시설팀" },  
    ...  
  ]  
}
```

```
]
}
```

3 민원 API

민원 목록 조회

```
GET /api/complaints
인증: 필요
```

역할에 따라 자동으로 필터링됩니다. - 사용자(C): 본인 민원만 - 처리자(E): 담당 카테고리 민원만 - 관리자(A): 전체 민원

쿼리 파라미터 (선택)

```
?category=미화
?status=접수전
?startDate=2026-04-01
?endDate=2026-04-30
?category=미화&status=완료&startDate=2026-04-01&endDate=2026-04-30
```

응답

```
{
  "complaints": [
    {
      "id": 1,
      "title": "1층 화장실 청소 요청",
      "content": "1층 화장실이 지저분합니다.",
      "category": "미화",
      "location": "본관 1층 화장실",
      "status": "접수전",
      "date": "2026-04-20 15:40:06.491771",
      "memberName": "홍길동",
      "memberDept": "총무팀"
    }
  ]
}
```

민원 상세 조회

```
GET /api/complaints/:id
인증: 필요
```

응답

```
{
  "complain": {
    "id": 1,
    "title": "1층 화장실 청소 요청",
    "content": "1층 화장실이 지저분합니다.",
    "category": "미화",
    "location": "본관 1층 화장실",
    "status": "접수전",
    "date": "2026-04-20 15:40:06.491771",
    "memberName": "홍길동",
    "memberDept": "총무팀"
  },
  "process": {
```

```
"result": "청소 완료하였습니다.",
"resultPerson": "김처리",
"resultDate": "2026-04-21 10:00:00"
},
"images": [
  { "id": 1, "url": "/uploads/complain/complain_xxx.jpg", "order": 1, "size": 12345 }
],
"processImages": [
  { "id": 1, "url": "/uploads/process/process_xxx.jpg", "order": 1, "size": 12345 }
]
}
```

process 가 null 이면 아직 처리되지 않은 민원입니다. 이미지 URL 접근: <http://localhost/uploads/complain/파일명>

민원 등록

POST /api/complaints
인증: 필요

요청 바디

```
{
  "category_id": 6,
  "title": "1층 화장실 청소 요청",
  "content": "1층 화장실이 지저분합니다.",
  "location": "본관 1층 화장실"
}
```

응답

```
{
  "message": "민원이 등록되었습니다.",
  "complain": {
    "id": 1,
    "title": "1층 화장실 청소 요청",
    "status": "접수전",
    "date": "2026-04-20 15:40:06"
  }
}
```

민원 등록 후 이미지가 있으면 POST /api/uploads/complain 으로 업로드해주세요.

민원 수정 (접수전만 가능)

PUT /api/complaints/:id
인증: 필요

요청 바디

```
{
  "category_id": 6,
  "title": "수정된 제목",
  "content": "수정된 내용",
  "location": "수정된 장소"
}
```

민원 삭제 (접수전만 가능)

```
DELETE /api/complaints/:id
인증: 필요
```

응답

```
{
  "message": "민원이 삭제되었습니다."
}
```

민원 상태 변경 (처리자/관리자)

```
PUT /api/complaints/:id/state
인증: 필요 (처리자/관리자만)
```

요청 바디

```
{
  "state": "A"
}
```

state 값: B (접수전), A (접수), P (진행중), D (완료)

민원 처리 등록 (처리자/관리자)

```
POST /api/complaints/:id/process
인증: 필요 (처리자/관리자만)
```

요청 바디

```
{
  "process_content": "청소 완료하였습니다."
}
```

처리 등록 시 민원 상태가 자동으로 완료(D)로 변경됩니다. 민원인에게 완료 알림이 자동 발송됩니다.

민원 엑셀 다운로드 (처리자/관리자)

```
GET /api/complaints/export
인증: 필요 (처리자/관리자만)
```

쿼리 파라미터 (선택)

```
?startDate=2026-04-01&endDate=2026-04-30
```

프론트 구현 예시

```
const handleExcelDownload = async () => {
  const token = localStorage.getItem('token');
  const response = await fetch(
    '/api/complaints/export?startDate=2026-04-01&endDate=2026-04-30',
    { headers: { Authorization: `Bearer ${token}` } }
  );
  const blob = await response.blob();
  const url = window.URL.createObjectURL(blob);
```

```
const a = document.createElement('a');
a.href = url;
const disposition = response.headers.get('Content-Disposition');
const filename = decodeURIComponent(disposition.split("filename*=UTF-8'")[1]);
a.download = filename;
a.click();
window.URL.revokeObjectURL(url);
};
```

4 이미지 업로드 API

민원 이미지 업로드

```
POST /api/uploads/complain
인증: 필요
Content-Type: multipart/form-data
```

요청 (form-data)

```
complain_id: 1
images: [파일1, 파일2, ...] // 최대 10장
```

프론트 구현 예시

```
const formData = new FormData();
formData.append('complain_id', complainId);
images.forEach(img => formData.append('images', img.file));

const response = await fetch('/api/uploads/complain', {
  method: 'POST',
  headers: { Authorization: `Bearer ${token}` },
  body: formData,
});
```

응답

```
{
  "message": "이미지가 업로드되었습니다.",
  "images": [
    {
      "complain_img_id": 1,
      "complain_img_url": "/uploads/complain/complain_xxx.jpg",
      "complain_img_order": 1,
      "complain_img_size": 12345
    }
  ]
}
```

민원 이미지 삭제

```
DELETE /api/uploads/complain/:id
인증: 필요
```

처리 이미지 업로드

```
POST /api/uploads/process
인증: 필요 (처리자/관리자만)
Content-Type: multipart/form-data
```

요청 (form-data)

```
process_id: 1
images: [파일1, 파일2, ...] // 최대 10장
```

5 공지사항 API

공지사항 목록 조회

```
GET /api/notices
인증: 필요
```

응답

```
{
  "notices": [
    {
      "id": 1,
      "title": "시스템 점검 안내",
      "category": "점검",
      "content": "점검 내용...",
      "author": "홍길동",
      "date": "2026-04-20 13:26:31"
    }
  ]
}
```

category 값: 안내 (G), 업데이트 (U), 점검 (F)

공지사항 상세 조회

```
GET /api/notices/:id
인증: 필요
```

공지사항 등록 (처리자/관리자)

```
POST /api/notices
인증: 필요 (처리자/관리자만)
```

요청 바디

```
{
  "notice_title": "시스템 점검 안내",
  "notice_category": "F",
  "notice_content": "점검 내용..."
}
```

notice_category: G (안내), U (업데이트), F (점검)

공지사항 수정 (작성자/관리자)

```
PUT /api/notices/:id
인증: 필요
```

공지사항 삭제 (작성자/관리자)

```
DELETE /api/notices/:id
```

인증: 필요

6 알림 API

알림 목록 조회

```
GET /api/notifications
```

인증: 필요

응답

```
{
  "notifications": [
    {
      "id": 1,
      "complainId": 3,
      "title": "1층 화장실 청소 요청",
      "content": "\"1층 화장실 청소 요청\"이(가) 완료 처리되었습니다.",
      "state": "완료",
      "read": false,
      "readAt": null,
      "time": "2026-04-20 16:22:26"
    }
  ],
  "unreadCount": 1
}
```

최근 7일치 알림만 반환됩니다. `unreadCount` 를 상단 벨 아이콘 배지에 표시해주세요.

읽지 않은 알림 수 조회 (벨 아이콘 배지용)

```
GET /api/notifications/unread-count
```

인증: 필요

응답

```
{
  "unreadCount": 3
}
```

알림 읽음 처리 (알림 클릭 시)

```
PUT /api/notifications/:id/read
```

인증: 필요

전체 읽음 처리

```
PUT /api/notifications/read-all
```

인증: 필요

7 회원 API

내 정보 수정 (비밀번호, 전화번호)

PUT /api/members/me
인증: 필요

요청 바디

```
{  
  "phone": "010-9999-8888"  
}
```

```
{  
  "password": "newpass1234!",  
  "phone": "010-9999-8888"  
}
```

비밀번호 변경 없이 전화번호만 수정할 수도 있어요.

처리자 프로필 조회 (민원 상세에서 처리자 이름 클릭 시)

GET /api/members/:id/profile
인증: 필요

응답

```
{  
  "profile": {  
    "name": "김처리",  
    "dept": "미화",  
    "phone": "010-1111-2222"  
  }  
}
```

회원 목록 조회 (관리자만)

GET /api/members
인증: 필요 (관리자만)

회원 승인 (관리자만)

PUT /api/members/:id/approve
인증: 필요 (관리자만)

권한 변경 (관리자만)

PUT /api/members/:id/role
인증: 필요 (관리자만)

요청 바디

```
{
  "role": "E"
}
```

role: c (사용자), E (처리자), A (관리자)

담당 카테고리 변경 (관리자만)

PUT /api/members/:id/dept
인증: 필요 (관리자만)

요청 바디

```
{
  "dept": "전체"
}
```

dept 값: 전체, 건축/영선, 의료장비, 기계/소방, 전기/통신, 보안, 미화

회원 탈퇴 처리 (관리자만)

DELETE /api/members/:id
인증: 필요 (관리자만)

필드명 매핑표

민원 필드명

백엔드 DB	API 응답 (프론트용)
complain_id	id
complain_title	title
complain_content	content
category_name	category
state (B/A/P/D)	status (접수전/접수/진행중/완료)
complain_at	date
process_content	result
process_by_name	resultPerson
process_at	resultDate

공지사항 필드명

백엔드 DB	API 응답 (프론트용)
notice_id	id
notice_title	title

백엔드 DB	API 응답 (프론트용)
notice_category (G/U/F)	category (안내/업데이트/점검)
notice_content	content
author_name	author
noticed_at	date

알림 필드명

백엔드 DB	API 응답 (프론트용)
push_id	id
complain_id	complainId
complain_title	title
push_content	content
is_read	read
read_at	readAt
push_at	time

 권한별 접근 가능 API

API	사용자(C)	처리자(E)	관리자(A)
로그인/회원가입	✓	✓	✓
민원 등록	✓	✓	✓
민원 목록 조회	본인만	담당 카테고리	전체
민원 수정/삭제	접수전 본인만	✗	✓
민원 상태 변경	✗	✓	✓
민원 처리 등록	✗	✓	✓
엑셀 다운로드	✗	✓	✓
공지사항 조회	✓	✓	✓
공지사항 등록/수정/삭제	✗	✓	✓
알림 조회	✓	✓	✓
내 정보 수정	✓	✓	✓
회원 관리	✗	✗	✓